

Laboratorium PK					
Rok akademicki	Rodzaj studiów	Kierunek	Prowadzący	Semestr	Sekcja
2025/2026	Dzienne	IPpp	LM	4	1

Sprawozdanie z projektu z przedmiotu
sieci komputerowe

1. Skład sekcji wraz z podziałem obowiązków w realizacji projektu

- **Krzysztof Bierzanek:** Analiza dostarczonego kodu bazowego symulatora, adaptacja istniejącego interfejsu graficznego (GUI) do pracy w architekturze sieciowej, wdrożenie blokad sprzężenia zwrotnego w kontrolkach (hermetyzacja za pomocą `QSignalBlocker`), obsługa płynnego przejścia między trybem sieciowym a lokalnym.
- **Igor Starnowski:** Zaprojektowanie protokołu komunikacyjnego, implementacja gniazd TCP (Klient/Serwer), realizacja binarnej serializacji istniejących obiektów symulacji do paczek danych, obsługa i pomiar opóźnień sieciowych (mechanizm Ping-Pong z automatycznym bezpiecznikiem).

2. Opis działania protokołu komunikacyjnego, serializacji i wykrywania gubienia pakietów

Protokół komunikacyjny i Serializacja

Komunikacja między węzłami (Regulatorem a Obiektem) odbywa się w oparciu o protokół TCP/IP. Zaprojektowano autorski protokół warstwy aplikacji, w którym każda ramka ma ściśle zdefiniowaną strukturę. Do zamiany gotowych obiektów np. `RegulatorPID`, `ModelARX` na ciąg bajtów wykorzystano mechanizm serializacji klasy `QDataStream`. Struktura pojedynczej ramki:

1. **Rozmiar ramki (4 bajty):** Typ `quint32`, całkowity rozmiar pakietu w bajtach.
2. **ID Typu Danych (1 bajt):** Typ `quint8` oparty na strukturze enum, identyfikujący zawartość (np. `0x02` dla ARX, `0x63` dla PING).
3. **Znacznik czasu (8 bajtów):** Typ `qint64`, przechowujący czas wygenerowania ramki w milisekundach od epoki systemowej.
4. **Dane (zmienna długość):** Zserializowany ładunek użyteczny (nastawy, parametry generatora).

Program ładuje strumień danych do obiektu `QByteArray`, rezerwując początkowo 4 bajty zerowe na rozmiar ramki, po czym na końcu procesu serializacji cofa wskaźnik i nadpisuje rezerwację rzeczywistym rozmiarem. Odbiorca używa buforowania – oczekuje w pętli `while(true)`, aż `bytesAvailable()` osiągnie zadeklarowany rozmiar, co gwarantuje odbiór pełnych, niesfragmentowanych ramek i chroni przed tzw. problemem nagłówka TCP.

Wykrywanie opóźnień i gubienia pakietów Ponieważ protokół TCP sprzętowo retransmituje zgubione pakiety w warstwie transportowej, dla aplikacji docelowej "gubienie pakietów" objawia się jako drastyczny wzrost opóźnienia. W dyskretnych układach automatyki buforowanie spóźnionych ramek prowadzi do całkowitej desynchronizacji symulacji. Wdrożono mechanizm **Application-level Ping-Pong**:

- Aplikacja wysyła synchroniczny puls (ramka `PAKIET_PING`) zawierający aktualny stempel czasowy.
- Odbiorca natychmiast odsyła ramkę (`PAKIET_PONG`), nie modyfikując stempla czasowego (mechanizm echa).

- Nadawca po odebraniu echa oblicza RTT (*Round Trip Time*) – niezależnie od synchronizacji zegarów na obu maszynach – i dzieli go przez 2, uzyskując precyzyjne opóźnienie transportowe.
- **Fail-safe:** Jeśli wykryte opóźnienie przekroczy wartość interwału symulacji, system uznaje połączenie za niestabilne, automatycznie zrywa gniazda TCP i przechodzi płynnie w "Tryb Lokalny". Symulacja jest kontynuowana na ostatnich pomyślnie odebranych nastawach bez ingerencji w generowanie wykresów.

3. Krótka historia rozwoju aplikacji o funkcjonalności sieciowe

- **Faza 1: Analiza kodu bazowego i przygotowanie architektury.** Zapoznanie się z dostarczonym, lokalnym silnikiem symulacji i wytypowanie punktów styku (sygnałów/slotów), które należało rozdzielić, aby umożliwić pracę w trybie rozproszonym Klient-Server.
- **Faza 2: Implementacja gniazd TCP i logiki połączeniowej.** Wprowadzenie do projektu nowych klas `KlientTCP` oraz `ServerTCP` opartych na `QTcpSocket` i `QTcpServer`. Stworzenie okna dialogowego do konfiguracji adresów IP i portów.
- **Faza 3: Projekt autorskiego protokołu i binarnej serializacji.** Zastąpienie lokalnego przekazywania wskaźników pakowaniem gotowych obiektów do postaci binarnej za pomocą `QDataStream`. Zbudowanie struktury ramki z nagłówkiem i identyfikatorem typu (enum).
- **Faza 4: Integracja sieci z istniejącym interfejsem.** Podpięcie oryginalnych zdarzeń GUI (`valueChanged`, `editingFinished`) pod funkcje wysyłające ramki sieciowe. Zsynchronizowanie kluczowego dla symulacji parametru *Interwał* po obu stronach sieci.
- **Faza 5: System Heartbeat i Fail-Safe (Zarządzanie opóźnieniami).** Wdrożenie mechanizmu "Ping-Pong" działającego w tle niezależnie od wymiany parametrów modelu. Napisanie logiki awaryjnego zrywania połączenia przy zbyt dużym opóźnieniu i płynnego powrotu symulacji do pracy na wartościach lokalnych.

4. Najistotniejsze napotkane trudności w realizacji tematu i sposoby ich rozwiązania

- **Pułapka czasu absolutnego przy mierzeniu Pingu:** Początkowo opóźnienie sieciowe liczono odejmując czas z odebranego pakietu od aktualnego czasu maszyny odbiorczej (*One-Way Delay*). Doprowadziło to do błędów (np. ujemny ping -5789 ms lub wartości rzędu 5794 ms). Zrozumiano, że zegary systemowe Windows na różnych maszynach są względem siebie asynchroniczne.
 - *Rozwiązanie:* Zmiana architektury na klasyczny system Ping-Pong (Echo). Obliczenia przesunięto z powrotem na maszynę nadawcy, która mierzy *Round Trip Time* wykorzystując wyłącznie swój własny zegar, a wynik dzieli przez 2.
- **Zjawisko Jitteru na łączach bezprzewodowych (Stress Test na 2.4 GHz):** W trakcie testów na dwóch komputerach (w sieci Wi-Fi 2.4 GHz) zaobserwowano nagle skoki opóźnień (np. do 70-280 ms) wynikające z naturalnych retransmisji warstwy fizycznej (zagłuszanie pasma radiowego).

- *Rozwiązanie:* Zjawisko to skutecznie psuło ciągłość wyliczania algorytmu sterowania i wymusiło wprowadzenie mechanizmu ochrony. Gdy opóźnienie przekroczy krytyczny próg (interwał symulacji), aplikacja odcina łączy i symuluje układ dalej w oparciu o pamięć podręczną (Tryb Lokalny).
- **Zapętlanie się sygnałów aktualizujących GUI (Nieskończona pętla nadawania):** Podczas odbierania nowej konfiguracji (np. interwału), funkcja aktualizująca kontrolkę interfejsu na nowo emitowała sprzętowy sygnał, co powodowało odesłanie tego samego pakietu z powrotem do nadawcy, zapychając gniazda TCP.
 - *Rozwiązanie:* Hermetyzacja funkcji odświeżających interfejs z wykorzystaniem klasy `QSignalBlocker(ui->kontrolka)`, co wycisza wewnętrzne sygnały komponentu na czas programowego przypisywania mu nowych wartości z sieci.

5. Zwięzłe podsumowanie czego nauczył się każdy członek sekcji

- **Krzysztof Bierzanek:** W ramach realizacji projektu nauczyłem się, jak trudno jest wziąć gotowy program działający na jednym komputerze i przerobić go tak, aby działał przez sieć, nie psując przy tym jego dotychczasowych funkcji. Poznałem w praktyce, jak duży wpływ mają realne parametry sieci domowych (Jitter, opóźnienie transportowe) na ciągłość dostarczania danych do algorytmu regulacji. Dodatkowo nauczyłem się, jak sprawnie pakować i wysyłać dane przez sieć z wykorzystaniem `QDataStream` oraz opanowałem zarządzanie cyklem życia sygnałów i slotów w Qt.
- **Igor Starnowski:** Rozbudowując istniejącą aplikację, zrozumiałem, jak optymalnie realizować architekturę zdarzeniową (*event-driven*) w warstwie sieciowej języka C++. Nauczyłem się budować system odporny na opóźnienia transportowe i gubienie pakietów TCP poprzez implementację mechanizmu fall-back (praca wyspowa w przypadku zerwania łączności). Znacząco pogłębiłem swoje umiejętności w bezpiecznej obsłudze asynchronicznych gniazd sieciowych oraz rozwiązywaniu specyficznych problemów środowiska Qt podczas integrowania nowych modułów z bazowym projektem.